

VM × File × I/O

文件内容怎样成为可映射物理页面，并在缺页、回写和 I/O 完成中保持身份

408 · OS-B04

Mother Interface

File Object + File Range ↔ Cached/Resident Page State

↔ Process VA Mapping

I/O request / completion ↔ Persistent Storage State

本 Bridge 专门连接“文件系统（内容身份）、虚拟内存（地址空间映射与驻留）与 I/O 子系统（设备传输与回写）”的三方交接契约；页表硬件翻译归入 X-B02，外设中断与 DMA 硬件归入 X-B03。

先定义接口两侧的词

- **File Object + File Range（文件对象 + 范围）**：文件系统对“这一页内容来自哪个持久对象的哪一段”的逻辑身份。经典 UNIX 可写成 inode/object + page index/offset，但真正 Owner 是 OS-06/07 的文件对象与 byte-to-block 关系。
- **Page Cache / Cached File Page（文件页缓存）**：OS 为 file-backed 内容维护的内存副本/驻留状态。Linux 的 address_space/folio/page 是工程表示；普通 buffered I/O 与 mmap 往往在这里汇合，但 direct I/O 等路径可以绕过部分缓存语义。
- **File-backed Mapping（文件后备映射）**：进程地址空间中的某段 VA 与文件对象范围建立关系；建立 mapping 不代表内容已经驻留。
- **Page-In / Fill（装入/填充）**：当所需文件内容当前没有可用内存副本时，VM/FS 需要产生存储请求并在完成后建立可用 resident state；是否由 DMA、interrupt 或 polling 完成属于 OS-05/X-B03 的下游机制。
- **Dirty + Writeback（脏状态与写回）**：内存中的 file-backed 内容比持久副本更新时，系统必须记录“有未持久化修改”，并在策略或显式 durability 请求要求时产生写回；谁触发、何时批量、何时才算 durable 由 OS-05/06/07 分责。

Owners 与职责边界

- **左侧 Owner (OS-06/07 文件系统)**：拥有 Inode、目录路径解析、逻辑块号到物理块号（LBN → PBN）的映射。
- **中间 Owner (OS-04 虚拟内存)**：拥有 VMA 范围、PTE 权限/有效位、缺页异常处理与物理页置换策略。
- **右侧 Owner (OS-05 I/O 系统)**：拥有 I/O 请求队列、电梯调度算法（SCAN/C-SCAN）、设备驱动与 DMA 控制。
- **本 Bridge Owns**：文件内容身份怎样与 cached/resident page state、进程 VA mapping 和 I/O request/completion 发生稳定 handoff；它不拥有任一侧内部的数据结构或回收/调度策略。

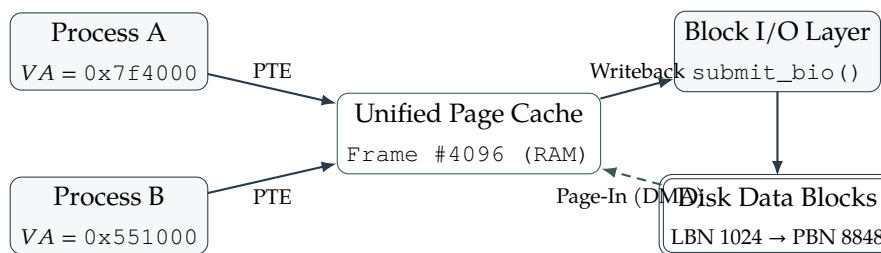
1 核心机制：同一内容要追踪三种“关系”，不是宣称三个身份完全等价

一段 file-backed 内容跨层时至少要同时保留三类关系：

1. **持久内容关系**：某个 File Object 的某个 byte/page range，决定数据在文件语义中“是谁”；
2. **缓存/驻留关系**：该内容当前是否已有可用的内存副本、它是否 dirty/writeback，以及谁引用这份 resident state；
3. **虚拟映射关系**：某个进程的 VA region/PTE 是否把这份内容暴露为可访问地址。

同一个 file range 可以暂时没有 resident page，也可以被多个进程映射到不同 VA；一个 resident page 还可能正处 I/O 中。Bridge 的价值就是防止把内容身份、驻留状态、虚拟地址压成一个对象。

2 母模型：统一页缓存、缺页装入与回写流转拓扑



3 双向路径推导：缺页/缓存未命中与脏页写回

3.1 路径 A：File-backed Fault / Fill

1. 进程访问 file-backed VA，硬件发现当前翻译/权限状态不足以完成访问并进入 fault 慢路径；
2. VM 根据 region/mapping 找到对应 File Object + File Range；
3. 检查是否已有可复用的 cached/resident 内容：
 - **Resident/Cache Hit**：建立或修复当前进程的映射关系，满足权限后重试原访问；
 - **Content Not Resident**：文件系统把 file range 翻译成所需存储请求，OS-05 接管 request/driver/device progress；当前 task 若采用阻塞路径，则通过 OS-B01 建立等待关系并让出 CPU；
4. I/O completion 先更新“目标内存内容已可用/请求已完成”等软件状态，再使相关等待者 Ready/Runnable；完成发现可以来自 interrupt、polling 或混合机制；
5. task 以后真正被调度运行时，VM 完成必要 mapping/fault return，并重试原访问。

3.2 路径 B：Dirty File Page → Writeback

1. 某次写入使内存中的 file-backed 内容比其持久副本更新，系统把相应缓存状态记为 dirty；硬件 dirty bit 可以帮助发现写入，但不是所有 OS 的唯一 dirty Owner；
2. 当 writeback policy、内存回收压力或显式 msync/fsync 一类接口要求推进持久状态时，FS/VM 形成对应文件范围的写回工作；

3. OS-05 把它转换为设备 request 并完成传输；请求可能被合并、拆分、排队，也不要求“一页恰好一次 DMA”；
4. completion 到达后，内核根据本次提交期间是否又发生新写入等状态，决定哪些内容可以视为不再 dirty/不再 writeback。若调用方请求的是 durability，还必须继续满足 OS-06/07 定义的持久化协议，不能把“设备请求完成”直接等同于“文件事务 durable”。

4 read()/write() vs mmap() 全景对比

对比维度	经典 read() / write()	内存映射 mmap()	性能与机制本质
数据路径	通过系统调用显式把 file range 与用户 buffer 之间的数据推进	建立 VA 到 file-backed 内存的映射，之后访问由普通 load/store + fault 慢路径驱动	两者通常都可复用文件页缓存；是否额外复制、是否命中 cache 取决于具体路径与实现
控制权入口	每次 read/write 调用都有 syscall 进入/返回路径；真正是否 block 还看数据是否立即可用	建立 mapping 需要 syscall，之后每次访问不必 syscall；首次/再次缺页仍会进入内核	“系统调用次数更少”不自动推出总体更快，page fault、TLB、局部性和访问模式都进入成本
地址空间状态	用户 buffer 与文件内容是两个显式对象	额外建立 file-backed VA region 与页表关系	mmap 把文件访问变成地址空间语义，因此要同时承担 mapping/lifetime 成本
共享语义	多进程共享取决于它们是否共享打开实例、共享内存或应用协议	MAP_SHARED 一类映射允许多个进程针对同一 file-backed 内容建立共享可见关系	“可见”不等于并发安全，也不等于修改已持久化到设备

Anti-Bridge：三个必须阻断的混淆

• Page Fault ≠ 文件读取失败：在文件映射区，首次缺页是按需加载数据的正常驱动机制，绝非错误。

• Hardware Cache (L1/L2/L3) ≠ OS Page Cache (RAM)：前者由 CPU 硬件自动管理，存放物理内存数据副本；后者由操作系统软件用 DRAM 管理磁盘文件页。

• mmap() 返回成功 ≠ 文件已读入内存：mmap() 建立的是 file-backed VA mapping；具体页面可以按需驻留，也可能因预读、已有 cache 或显式 populate 等原因提前存在，不能把 mapping creation 与 residency 画等号。

30 秒极速调用协议

做题遇到“mmap 读写、Page Cache 共享、脏页回写与 I/O 路径”时，严格按三层追踪：

VA Mapping 对应哪个 File Range?

内容是否 Resident/Cached?

需要时提交 I/O 并等待 Completion

Dirty 与 Durable 状态